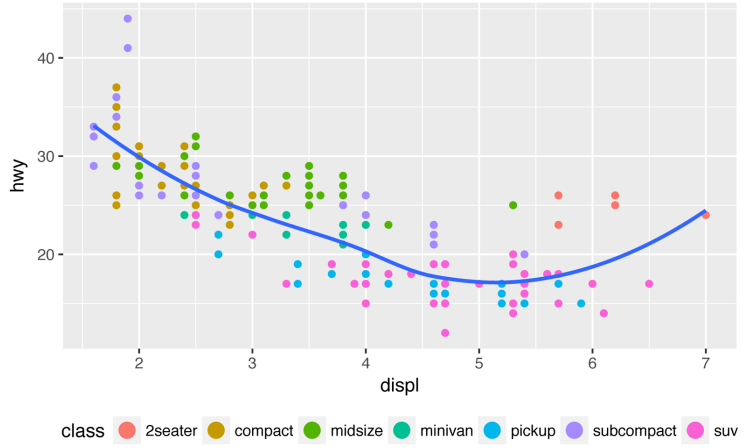
You can also use `legend.position = "none"` to suppress the display of the legend altogether.

To control the display of individual legends, use `guides()` along with `guide_legend()` or `guide_colorbar()`. The following example shows two important settings: controlling the number of rows the legend uses with `nrow`, and overriding one of the aesthetics to make the points bigger. This is particularly useful if you have used a low `alpha` to display many points on a plot:

```
ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(color = class)) +
  geom_smooth(se = FALSE) +
  theme(legend.position = "bottom") +
  guides(
    color = guide_legend(
      nrow = 1,
      override.aes = list(size = 4)
    )
  )
#> `geom_smooth()` using method = 'loess'
```



Replacing a Scale

Instead of just tweaking the details a little, you can replace the scale altogether. There are two types of scales you're most likely to want to switch out: continuous position scales and color scales. Fortunately, the same principles apply to all the other aesthetics, so once you've mastered position and color, you'll be able to quickly pick up other scale replacements.

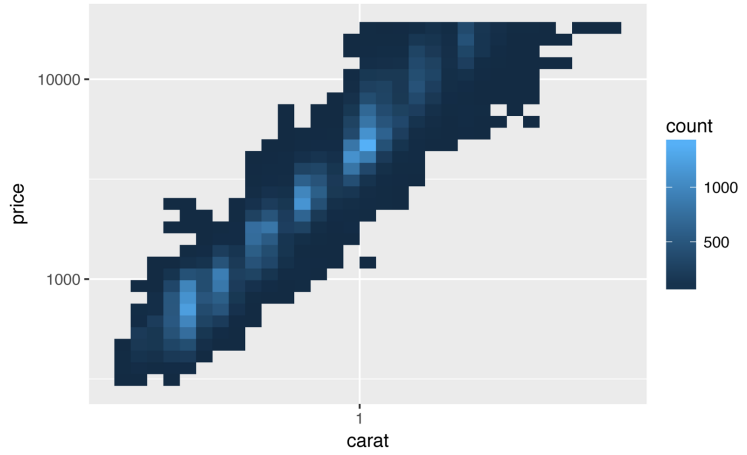
It's very useful to plot transformations of your variable. For example, as we've seen in "Why Are Low-Quality Diamonds More Expensive?" on page 376, it's easier to see the precise relationship between `carat` and `price` if we log-transform them:

```
ggplot(diamonds, aes(carat, price)) +
  geom_bin2d()

ggplot(diamonds, aes(log10(carat), log10(price))) +
  geom_bin2d()
```

However, the disadvantage of this transformation is that the axes are now labeled with the transformed values, making it hard to interpret the plot. Instead of doing the transformation in the aesthetic mapping, we can instead do it with the scale. This is visually identical, except the axes are labeled on the original data scale:

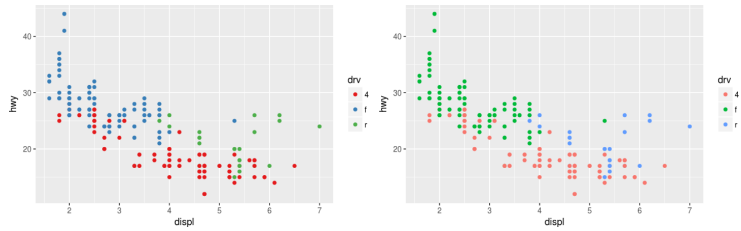
```
ggplot(diamonds, aes(carat, price)) +
  geom_bin2d() +
  scale_x_log10() +
  scale_y_log10()
```



Another scale that is frequently customized is color. The default categorical scale picks colors that are evenly spaced around the color wheel. Useful alternatives are the ColorBrewer scales, which have been hand-tuned to work better for people with common types of color blindness. The following two plots look similar, but there is enough difference in the shades of red and green that the dots on the right can be distinguished even by people with red-green color blindness:

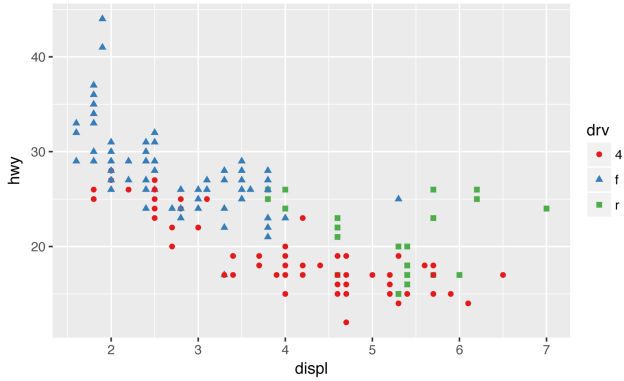
```
ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(color = drv))

ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(color = drv)) +
  scale_color_brewer(palette = "Set1")
```



Don't forget simpler techniques. If there are just a few colors, you can add a redundant shape mapping. This will also help ensure your plot is interpretable in black and white:

```
ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(color = drv, shape = drv)) +
  scale_color_brewer(palette = "Set1")
```

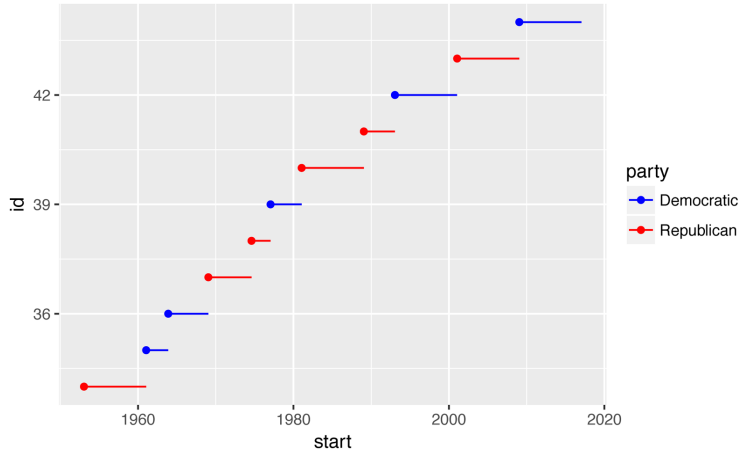


The ColorBrewer scales are documented online at *http://colorbrewer2.org/* and made available in R via the **RColorBrewer** package, by Erich Neuwirth. Figure 22-2 shows the complete list of all palettes. The sequential (top) and diverging (bottom) palettes are particularly useful if your categorical values are ordered, or have a "middle." This often arises if you've used `cut()` to make a continuous variable into a categorical variable.

Figure 22-2. All ColorBrewer scales

When you have a predefined mapping between values and colors, use `scale_color_manual()`. For example, if we map presidential party to color, we want to use the standard mapping of red for Republicans and blue for Democrats:

```
presidential %>%
  mutate(id = 33 + row_number()) %>%
  ggplot(aes(start, id, color = party)) +
    geom_point() +
    geom_segment(aes(xend = end, yend = id)) +
    scale_colour_manual(
      values = c(Republican = "red", Democratic = "blue")
    )
```



For continuous color, you can use the built-in `scale_color_gradient()` or `scale_fill_gradient()`. If you have a diverging scale, you can use `scale_color_gradient2()`. That allows you to give, for example, positive and negative values different colors. That's sometimes also useful if you want to distinguish points above or below the mean.

Another option is `scale_color_viridis()` provided by the **viridis** package. It's a continuous analog of the categorical ColorBrewer scales. The designers, Nathaniel Smith and Stéfan van der Walt, carefully tailored a continuous color scheme that has good perceptual properties. Here's an example from the **viridis** vignette:

```
df <- tibble(
  x = rnorm(10000),
  y = rnorm(10000)
```

```
)
ggplot(df, aes(x, y)) +
  geom_hex() +
  coord_fixed()
#> Loading required package: methods

ggplot(df, aes(x, y)) +
  geom_hex() +
  viridis::scale_fill_viridis() +
  coord_fixed()
```

Note that all color scales come in two varieties: `scale_color_x()` and `scale_fill_x()` for the `color` and `fill` aesthetics, respectively (the color scales are available in both UK and US spellings).

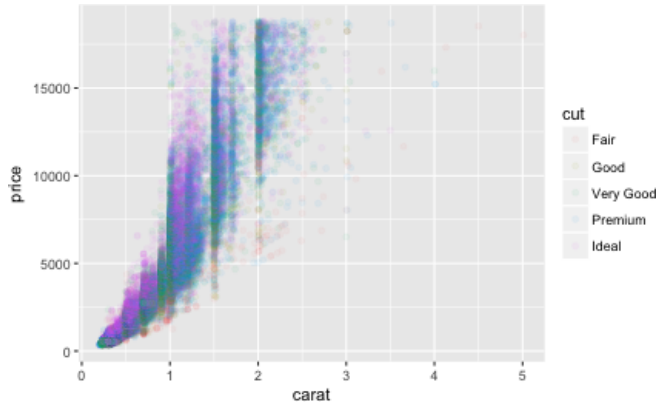
Exercises

1. Why doesn't the following code override the default scale?

   ```
   ggplot(df, aes(x, y)) +
     geom_hex() +
     scale_color_gradient(low = "white", high = "red") +
     coord_fixed()
   ```

2. What is the first argument to every scale? How does it compare to `labs()`?
3. Change the display of the presidential terms by:
   a. Combining the two variants shown above.
   b. Improving the display of the y-axis.
   c. Labeling each term with the name of the president.
   d. Adding informative plot labels.
   e. Placing breaks every four years (this is trickier than it seems!).
4. Use `override.aes` to make the legend on the following plot easier to see:

   ```
   ggplot(diamonds, aes(carat, price)) +
     geom_point(aes(color = cut), alpha = 1/20)
   ```



Zooming

There are three ways to control the plot limits:

- Adjusting what data is plotted
- Setting the limits in each scale
- Setting `xlim` and `ylim` in `coord_cartesian()`

To zoom in on a region of the plot, it's generally best to use `coord_cartesian()`. Compare the following two plots:

```
ggplot(mpg, mapping = aes(displ, hwy)) +
  geom_point(aes(color = class)) +
  geom_smooth() +
  coord_cartesian(xlim = c(5, 7), ylim = c(10, 30))

mpg %>%
  filter(displ >= 5, displ <= 7, hwy >= 10, hwy <= 30) %>%
  ggplot(aes(displ, hwy)) +
  geom_point(aes(color = class)) +
  geom_smooth()
```

You can also set the limits on individual scales. Reducing the limits is basically equivalent to subsetting the data. It is generally more useful if you want *expand* the limits, for example, to match scales across different plots. For example, if we extract two classes of cars and plot them separately, it's difficult to compare the plots because all three scales (the x-axis, the y-axis, and the color aesthetic) have different ranges:

```
suv <- mpg %>% filter(class == "suv")
compact <- mpg %>% filter(class == "compact")

ggplot(suv, aes(displ, hwy, color = drv)) +
  geom_point()

ggplot(compact, aes(displ, hwy, color = drv)) +
  geom_point()
```

One way to overcome this problem is to share scales across multiple plots, training the scales with the `limits` of the full data:

```
x_scale <- scale_x_continuous(limits = range(mpg$displ))
y_scale <- scale_y_continuous(limits = range(mpg$hwy))
col_scale <- scale_color_discrete(limits = unique(mpg$drv))

ggplot(suv, aes(displ, hwy, color = drv)) +
  geom_point() +
  x_scale +
  y_scale +
  col_scale

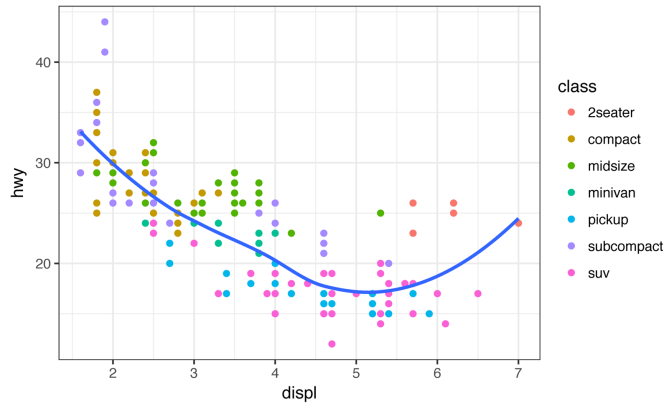
ggplot(compact, aes(displ, hwy, color = drv)) +
  geom_point() +
  x_scale +
  y_scale +
  col_scale
```

In this particular case, you could have simply used faceting, but this technique is useful more generally if, for instance, you want to spread plots over multiple pages of a report.

Themes

Finally, you can customize the nondata elements of your plot with a theme:

```
ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(color = class)) +
  geom_smooth(se = FALSE) +
  theme_bw()
```



ggplot2 includes eight themes by default, as shown in Figure 22-3. Many more are included in add-on packages like **ggthemes**, by Jeffrey Arnold.

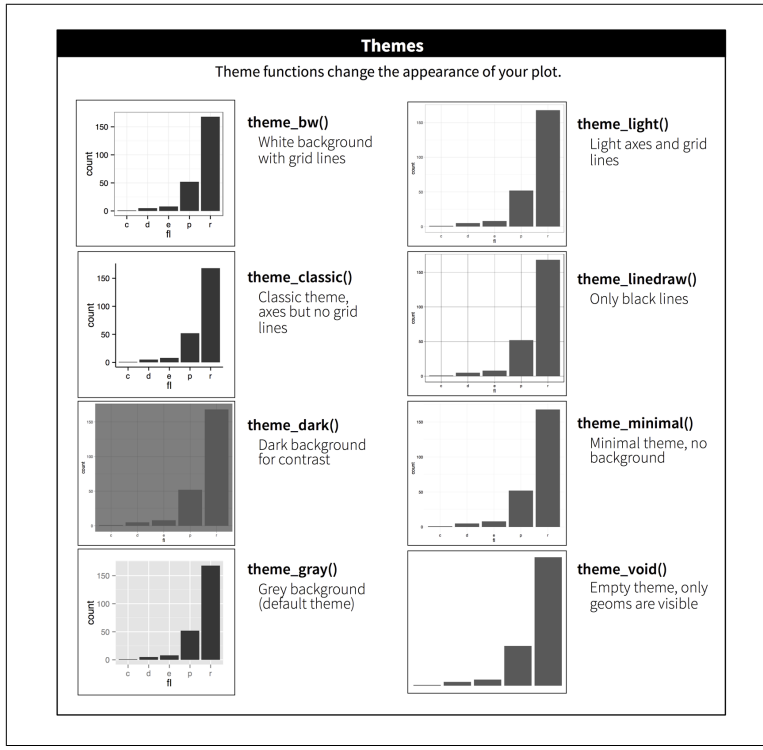


Figure 22-3. The eight themes built into ggplot2

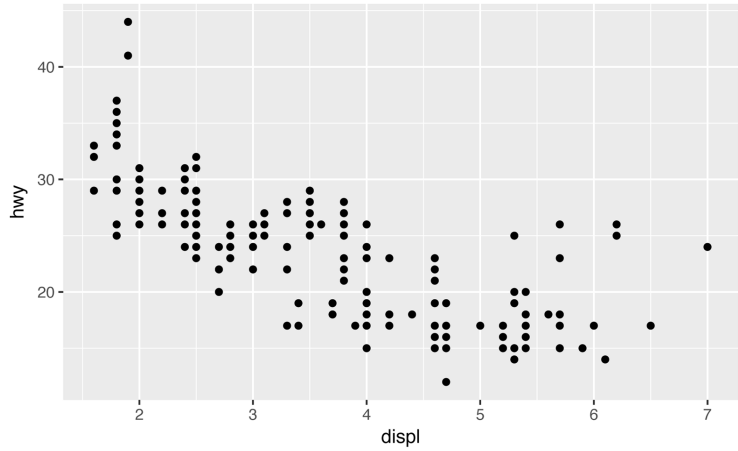
Many people wonder why the default theme has a gray background. This was a deliberate choice because it puts the data forward while still making the grid lines visible. The white grid lines are visible (which is important because they significantly aid position judgments), but they have little visual impact and we can easily tune them out. The gray background gives the plot a similar typographic color to the text, ensuring that the graphics fit in with the flow of a document without jumping out with a bright white background. Finally, the gray background creates a continuous field of color, which ensures that the plot is perceived as a single visual entity.

It's also possible to control individual components of each theme, like the size and color of the font used for the y-axis. Unfortunately, this level of detail is outside the scope of this book, so you'll need to read the **ggplot2** book for the full details. You can also create your own themes, if you are trying to match a particular corporate or journal style.

Saving Your Plots

There are two main ways to get your plots out of R and into your final write-up: `ggsave()` and **knitr**. `ggsave()` will save the most recent plot to disk:

```
ggplot(mpg, aes(displ, hwy)) + geom_point()
```




```
ggsave("my-plot.pdf")
#> Saving 6 x 3.71 in image
```

If you don't specify the `width` and `height` they will be taken from the dimensions of the current plotting device. For reproducible code, you'll want to specify them.

Generally, however, I think you should be assembling your final reports using R Markdown, so I want to focus on the important code chunk options that you should know about for graphics. You can learn more about `ggsave()` in the documentation.

Figure Sizing

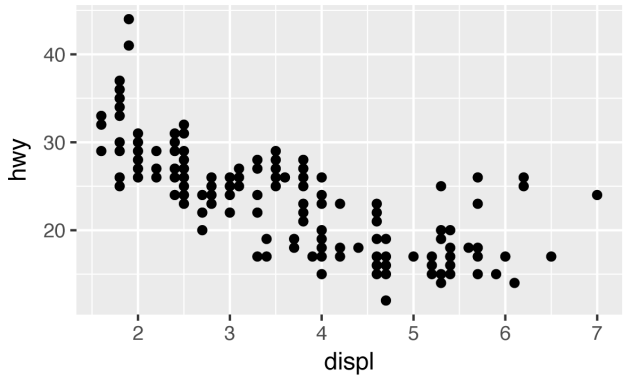
The biggest challenge of graphics in R Markdown is getting your figures the right size and shape. There are five main options that control figure sizing: `fig.width`, `fig.height`, `fig.asp`, `out.width`, and `out.height`. Image sizing is challenging because there are two sizes (the size of the figure created by R and the size at which it is inserted in the output document), and multiple ways of specifying the size (i.e., height, width, and aspect ratio: pick two of three).

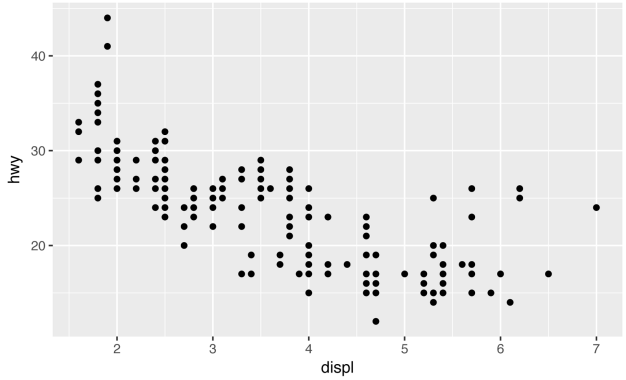
I only ever use three of the five options:

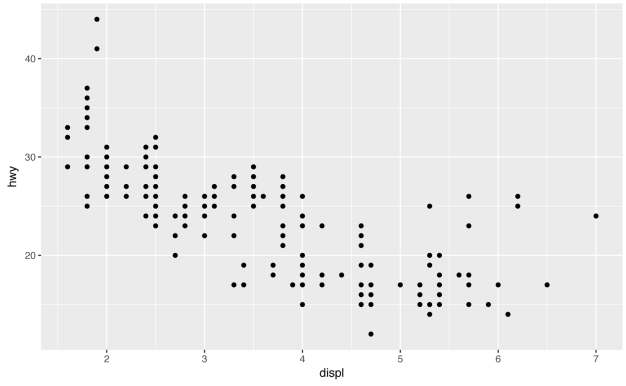
- I find it most aesthetically pleasing for plots to have a consistent width. To enforce this, I set `fig.width = 6` (6") and `fig.asp = 0.618` (the golden ratio) in the defaults. Then in individual chunks, I only adjust `fig.asp`.
- I control the output size with `out.width` and set it to a percentage of the line width). I default to `out.width = "70%"` and `fig.align = "center"`. That give plots room to breathe, without taking up too much space.
- To put multiple plots in a single row I set the `out.width` to `50%` for two plots, `33%` for three plots, or `25%` to four plots, and set `fig.align = "default"`. Depending on what I'm trying to illustrate (e.g., show data or show plot variations), I'll also tweak `fig.width`, as discussed next.

If you find that you're having to squint to read the text in your plot, you need to tweak `fig.width`. If `fig.width` is larger than the size the figure is rendered in the final doc, the text will be too small; if `fig.width` is smaller, the text will be too big. You'll often need to do a little experimentation to figure out the right ratio between the `fig.width` and the eventual width in your document. To illustrate

the principle, the following three plots have `fig.width` of 4, 6, and 8, respectively:







If you want to make sure the font size is consistent across all your figures, whenever you set `out.width`, you'll also need to adjust `fig.width` to maintain the same ratio with your default `out.width`. For example, if your default `fig.width` is 6 and `out.width` is 0.7, when you set `out.width = "50%"` you'll need to set `fig.width` to 4.3 (6 * 0.5 / 0.7).

Other Important Options

When mingling code and text, like I do in this book, I recommend setting `fig.show = "hold"` so that plots are shown after the code. This has the pleasant side effect of forcing you to break up large blocks of code with their explanations.

To add a caption to the plot, use `fig.cap`. In R Markdown this will change the figure from inline to "floating."

If you're producing PDF output, the default graphics type is PDF. This is a good default because PDFs are high-quality vector graphics. However, they can produce very large and slow plots if you are displaying thousands of points. In that case, set `dev = "png"` to force the use of PNGs. They are slightly lower quality, but will be much more compact.

It's a good idea to name code chunks that produce figures, even if you don't routinely label other chunks. The chunk label is used to generate the filename of the graphic on disk, so naming your chunks makes it much easier to pick out plots and reuse them in other circumstances (i.e., if you want to quickly drop a single plot into an email or a tweet).

Learning More

The absolute best place to learn more is the **ggplot2** book: *ggplot2: Elegant graphics for data analysis*. It goes into much more depth about the underlying theory, and has many more examples of how to combine the individual pieces to solve practical problems. Unfortunately, the book is not available online for free, although you can find the source code at *https://github.com/hadley/ggplot2-book*.

Another great resource is the **ggplot2** extensions guide. This site lists many of the packages that extend **ggplot2** with new geoms and scales. It's a great place to start if you're trying to do something that seems hard with **ggplot2**.

CHAPTER 23

R Markdown Formats

Introduction

So far you've seen R Markdown used to produce HTML documents. This chapter gives a brief overview of some of the many other types of output you can produce with R Markdown. There are two ways to set the output of a document:

1. Permanently, by modifying the the YAML header:

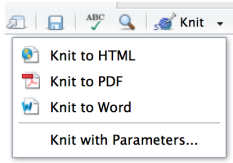
   ```
   title: "Viridis Demo"
   output: html_document
   ```

2. Transiently, by calling `rmarkdown::render()` by hand:

   ```
   rmarkdown::render(
     "diamond-sizes.Rmd",
     output_format = "word_document"
   )
   ```

   This is useful if you want to programmatically produce multiple types of output.

RStudio's knit button renders a file to the first format listed in its `output` field. You can render to additional formats by clicking the drop-down menu beside the knit button.



Output Options

Each output format is associated with an R function. You can either write `foo` or `pkg::foo`. If you omit `pkg`, the default is assumed to be **rmarkdown**. It's important to know the name of the function that makes the output because that's where you get help. For example, to figure out what parameters you can set with `html_document`, look at `?rmarkdown:html_document()`.

To override the default parameter values, you need to use an expanded `output` field. For example, if you wanted to render an `html_document` with a floating table of contents, you'd use:

```
output:
  html_document:
    toc: true
    toc_float: true
```

You can even render to multiple outputs by supplying a list of formats:

```
output:
  html_document:
    toc: true
    toc_float: true
  pdf_document: default
```

Note the special syntax if you don't want to override any of the default options.

Documents

The previous chapter focused on the default `html_document` output. There are number of basic variations on that theme, generating different types of documents:

- `pdf_document` makes a PDF with LaTeX (an open source document layout system), which you'll need to install. RStudio will prompt you if you don't already have it.

- `word_document` for Microsoft Word documents (*.docx*).
- `odt_document` for OpenDocument Text documents (*.odt*).
- `rtf_document` for Rich Text Format (*.rtf*) documents.
- `md_document` for a Markdown document. This isn't typically useful by itself, but you might use it if, for example, your corporate CMS or lab wiki uses Markdown.
- `github_document` is a tailored version of `md_document` designed for sharing on GitHub.

Remember, when generating a document to share with decision makers, you can turn off the default display of code by setting global options in the setup chunk:

```
knitr::opts_chunk$set(echo = FALSE)
```

For `html_documents` another option is to make the code chunks hidden by default, but visible with a click:

```
output:
  html_document:
    code_folding: hide
```

Notebooks

A notebook, `html_notebook`, is a variation on an `html_document`. The rendered outputs are very similar, but the purpose is different. An `html_document` is focused on communicating with decision makers, while a notebook is focused on collaborating with other data scientists. These different purposes lead to using the HTML output in different ways. Both HTML outputs will contain the fully rendered output, but the notebook also contains the full source code. That means you can use the *.nb.html* generated by the notebook in two ways:

- You can view it in a web browser, and see the rendered output. Unlike `html_document`, this rendering always includes an embedded copy of the source code that generated it.
- You can edit it in RStudio. When you open an *.nb.html* file, RStudio will automatically re-create the *.Rmd* file that generated it. In the future, you can also include supporting files (e.g., *.csv* data files), which will be automatically extracted when needed.

Emailing *.nb.html* files is a simple way to share analyses with your colleagues. But things will get painful as soon as they want to make changes. If this starts to happen, it's a good time to learn Git and GitHub. Learning Git and GitHub is definitely painful at first, but the collaboration payoff is huge. As mentioned earlier, Git and GitHub are outside the scope of the book, but there's one tip that's useful if you're already using them: use both `html_notebook` and `github_document` outputs:

```
output:
  html_notebook: default
  github_document: default
```

`html_notebook` gives you a local preview, and a file that you can share via email. `github_document` creates a minimal MD file that you can check into Git. You can easily see how the results of your analysis (not just the code) change over time, and GitHub will render it for you nicely online.

Presentations

You can also use R Markdown to produce presentations. You get less visual control than with a tool like Keynote or PowerPoint, but automatically inserting the results of your R code into a presentation can save a huge amount of time. Presentations work by dividing your content into slides, with a new slide beginning at each first (`#`) or second (`##`) level header. You can also insert a horizontal rule (`***`) to create a new slide without a header.

R Markdown comes with three presentations formats built in:

`ioslides_presentation`
: HTML presentation with ioslides.

`slidy_presentation`
: HTML presentation with W3C Slidy.

`beamer_presentation`
: PDF presentation with LaTeX Beamer.

Two other popular formats are provided by packages:

`revealjs::revealjs_presentation`
: HTML presentation with reveal.js. Requires the **revealjs** package.

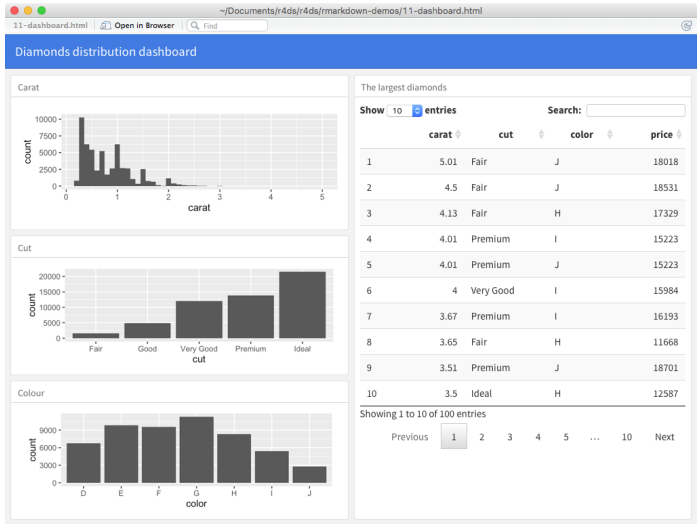
`rmdshower`
: Provides a wrapper around the **shower** presentation engine.

Dashboards

Dashboards are a useful way to communicate large amounts of information visually and quickly. **flexdashboard** makes it particularly easy to create dashboards using R Markdown and a convention for how the headers affect the layout:

- Each level 1 header (#) begins a new page in the dashboard.
- Each level 2 header (##) begins a new column.
- Each level 3 header (###) begins a new row.

For example, you can produce this dashboard:



Using this code:

```
---
title: "Diamonds distribution dashboard"
output: flexdashboard::flex_dashboard
---

```{r setup, include = FALSE}
library(ggplot2)
library(dplyr)
```
```

````
knitr::opts_chunk$set(fig.width = 5, fig.asp = 1/3)
```

## Column 1

### Carat

```{r}
ggplot(diamonds, aes(carat)) + geom_histogram(binwidth = 0.1)
```

### Cut

```{r}
ggplot(diamonds, aes(cut)) + geom_bar()
```

### Color

```{r}
ggplot(diamonds, aes(color)) + geom_bar()
```

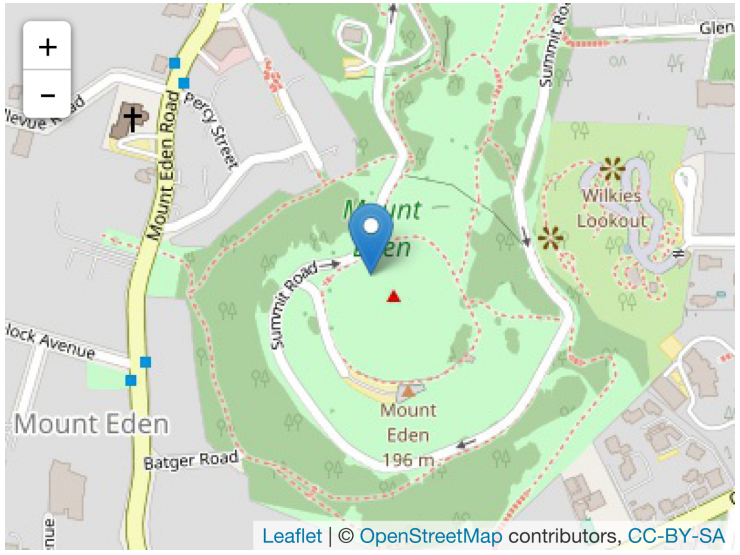
## Column 2

### The largest diamonds

```{r}
diamonds %>%
  arrange(desc(carat)) %>%
  head(100) %>%
  select(carat, cut, color, price) %>%
  DT::datatable()
```
````

flexdashboard also provides simple tools for creating sidebars, tabsets, value boxes, and gauges. To learn more about **flexdashboard** visit *http://rmarkdown.rstudio.com/flexdashboard/*.

Interactivity

Any HTML format (document, notebook, presentation, or dashboard) can contain interactive components.

htmlwidgets

HTML is an interactive format, and you can take advantage of that interactivity with *htmlwidgets*, R functions that produce interactive HTML visualizations. For example, take the following *leaflet* map. If
````

you're viewing this page on the web, you can drag the map around, zoom in and out, etc. You obviously can't do that on a book, so **rmarkdown** automatically inserts a static screenshot for you:

```
library(leaflet)
leaflet() %>%
  setView(174.764, -36.877, zoom = 16) %>%
  addTiles() %>%
  addMarkers(174.764, -36.877, popup = "Maungawhau")
```



The great thing about htmlwidgets is that you don't need to know anything about HTML or JavaScript to use them. All the details are wrapped inside the package, so you don't need to worry about it.

There are many packages that provide htmlwidgets, including:

- **dygraphs** for interactive time series visualizations.
- **DT** for interactive tables.
- **rthreejs** for interactive 3D plots.
- **DiagrammeR** for diagrams (like flow charts and simple node-link diagrams).

To learn more about htmlwidgets and see a more complete list of packages that provide them, visit *http://www.htmlwidgets.org/*.

Shiny

htmlwidgets provide *client-side* interactivity—all the interactivity happens in the browser, independently of R. On one hand, that's great because you can distribute the HTML file without any connection to R. However, that fundamentally limits what you can do to things that have been implemented in HTML and JavaScript. An alternative approach is to use **Shiny**, a package that allows you to create interactivity using R code, not JavaScript.

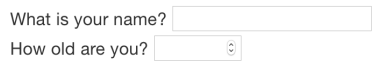
To call **Shiny** code from an R Markdown document, add `runtime: shiny` to the header:

```
title: "Shiny Web App"
output: html_document
runtime: shiny
```

Then you can use the "input" functions to add interactive components to the document:

```
library(shiny)

textInput("name", "What is your name?")
numericInput("age", "How old are you?", NA, min = 0, max = 150)
```

You can then refer to the values with `input$name` and `input$age`, and the code that uses them will be automatically rerun whenever they change.



I can't show you a live **Shiny** app here because **Shiny** interactions occur on the *server side*. This means you can write interactive apps without knowing JavaScript, but it means that you need a server to run it on. This introduces a logistical issue: **Shiny** apps need a **Shiny** server to be run online. When you run **Shiny** apps on your own computer, **Shiny** automatically sets up a **Shiny** server for you, but you need a public-facing **Shiny** server if you want to publish this sort of interactivity online. That's the fundamental trade-off of **Shiny**: you can do anything in a **Shiny** document that you can do in R, but it requires someone to be running R.

Learn more about **Shiny** at *http://shiny.rstudio.com/*.

Websites

With a little additional infrastructure you can use R Markdown to generate a complete website:

- Put your *.Rmd* files in a single directory. *index.Rmd* will become the home page.
- Add a YAML file named *_site.yml* that provides the navigation for the site. For example:

```
name: "my-website"
navbar:
  title: "My Website"
  left:
    - text: "Home"
      href: index.html
    - text: "Viridis Colors"
      href: 1-example.html
    - text: "Terrain Colors"
      href: 3-inline.html
```

Execute `rmarkdown::render_site()` to build *_site*, a directory of files ready to deploy as a standalone static website, or if you use an RStudio Project for your website directory. RStudio will add a Build tab to the IDE that you can use to build and preview your site.

Read more at *http://bit.ly/RMarkdownWebsites*.

Other Formats

Other packages provide even more output formats:

- The **bookdown** package makes it easy to write books, like this one. To learn more, read *Authoring Books with R Markdown*, by Yihui Xie, which is, of course, written in bookdown. Visit *http://www.bookdown.org* to see other bookdown books written by the wider R community.
- The **prettydoc** package provides lightweight document formats with a range of attractive themes.
- The **rticles** package compiles a selection of formats tailored for specific scientific journals.

See *http://rmarkdown.rstudio.com/formats.html* for a list of even more formats. You can also create your own by following the instructions at *http://bit.ly/CreatingNewFormats*.

Learning More

To learn more about effective communication in these different formats I recommend the following resources:

- To improve your presentation skills, I recommend *Presentation Patterns* by Neal Ford, Matthew McCollough, and Nathaniel Schutta. It provides a set of effective patterns (both low- and high-level) that you can apply to improve your presentations.
- If you give academic talks, I recommend reading the *Leek group guide to giving talks*.
- I haven't taken it myself, but I've heard good things about Matt McGarrity's online course on public speaking.
- If you are creating a lot of dashboards, make sure to read Stephen Few's *Information Dashboard Design: The Effective Visual Communication of Data*. It will help you create dashboards that are truly useful, not just pretty to look at.
- Effectively communicating your ideas often benefits from some knowledge of graphic design. *The Non-Designer's Design Book* is a great place to start.

CHAPTER 24
R Markdown Workflow

Earlier, we discussed a basic workflow for capturing your R code where you work interactively in the *console*, then capture what works in the *script editor*. R Markdown brings together the console and the script editor, blurring the lines between interactive exploration and long-term code capture. You can rapidly iterate within a chunk, editing and re-executing with Cmd/Ctrl-Shift-Enter. When you're happy, you move on and start a new chunk.

R Markdown is also important because it so tightly integrates prose and code. This makes it a great *analysis notebook* because it lets you develop code and record your thoughts. An analysis notebook shares many of the same goals as a classic lab notebook in the physical sciences. It:

- Records what you did and why you did it. Regardless of how great your memory is, if you don't record what you do, there will come a time when you have forgotten important details. Write them down so you don't forget!
- Supports rigorous thinking. You are more likely to come up with a strong analysis if you record your thoughts as you go, and continue to reflect on them. This also saves you time when you eventually write up your analysis to share with others.
- Helps others understand your work. It is rare to do data analysis by yourself, and you'll often be working as part of a team. A lab notebook helps you share not only what you've done, but why you did it with your colleagues or lab mates.

Much of the good advice about using lab notebooks effectively can also be translated to analysis notebooks. I've drawn on my own experiences and Colin Purrington's advice on lab notebooks (*http://colinpurrington.com/tips/lab-notebooks*) to come up with the following tips:

- Ensure each notebook has a descriptive title, an evocative filename, and a first paragraph that briefly describes the aims of the analysis.
- Use the YAML header date field to record the date you started working on the notebook:

  ```
  date: 2016-08-23
  ```

  Use ISO8601 YYYY-MM-DD format so that's there no ambiguity. Use it even if you don't normally write dates that way!
- If you spend a lot of time on an analysis idea and it turns out to be a dead end, don't delete it! Write up a brief note about why it failed and leave it in the notebook. That will help you avoid going down the same dead end when you come back to the analysis in the future.
- Generally, you're better off doing data entry outside of R. But if you do need to record a small snippet of data, clearly lay it out using `tibble::tribble()`.
- If you discover an error in a data file, never modify it directly, but instead write code to correct the value. Explain why you made the fix.
- Before you finish for the day, make sure you can knit the notebook (if you're using caching, make sure to clear the caches). That will let you fix any problems while the code is still fresh in your mind.
- If you want your code to be reproducible in the long run (i.e., so you can come back to run it next month or next year), you'll need to track the versions of the packages that your code uses. A rigorous approach is to use **packrat**, which stores packages in your project directory, or **checkpoint**, which will reinstall packages available on a specified date. A quick and dirty hack is to include a chunk that runs `sessionInfo()`—that won't let you easily re-create your packages as they are today, but at least you'll know what they were.

- You are going to create many, many, many analysis notebooks over the course of your career. How are you going to organize them so you can find them again in the future? I recommend storing them in individual projects, and coming up with a good naming scheme.

Index

E

F

G

H

N

O

P

Q

R

S

T

U

V

W

X

Y

Z

About the Authors

Hadley Wickham is Chief Scientist at RStudio and a member of the R Foundation. He builds tools (both computational and cognitive) that make data science easier, faster, and more fun. His work includes packages for data science (the tidyverse: ggplot2, dplyr, tidyr, purrr, readr, ...), and principled software development (roxygen2, testthat, devtools). He is also a writer, educator, and frequent speaker promoting the use of R for data science. Learn more on his website, *http://hadley.nz*.

Garrett Grolemund is a statistician, teacher, and R developer who works for RStudio. He wrote the well-known lubridate R package and is the author of *Hands-On Programming with R* (O'Reilly).

Garrett is a popular R instructor at *DataCamp.com* and *oreilly.com/safari*, and has been invited to teach R and Data Science at many companies, including Google, eBay, Roche, and more. At RStudio, Garrett develops webinars, workshops, and an acclaimed series of cheat sheets for R.

Colophon

The animal on the cover of *R for Data Science* is the kakapo (*Strigops habroptilus*). Also known as the owl parrot, the kakapo is a large flightless bird native to New Zealand. Adult kakapos can grow up to 64 centimeters in height and 4 kilograms in weight. Their feathers are generally yellow and green, although there is significant variation between individuals. Kakapos are nocturnal and use their robust sense of smell to navigate at night. Although they cannot fly, kakapos have strong legs that enable them to run and climb much better than most birds.

The name kakapo comes from the language of the native Maori people of New Zealand. Kakapos were an important part of Maori culture, both as a food source and as a part of Maori mythology. Kakapo skin and feathers were also used to make cloaks and capes.

Due to the introduction of predators to New Zealand during European colonization, kakapos are now critically endangered, with less than 200 individuals currently living. The government of New Zealand has been actively attempting to revive the kakapo population by providing special conservation zones on three predator-free islands.

Many of the animals on O'Reilly covers are endangered; all of them are important to the world. To learn more about how you can help, go to *animals.oreilly.com*.

The cover image is from *Wood's Animate Creations*. The cover fonts are URW Typewriter and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.